Software Validation and Testing Tools (SVTT)

PRACTICAL SOLUTIONS — EXAM READY GUIDE

All 10 Practicals with Commands, Code & Conclusions

How to Read This Guide

`lightblue` **Label**	**Meaning**
`COMMAND`	A terminal/shell command you type in Command Prompt or Terminal
`CODE`	Python script you write in a .py file and run with Python
`CONCEPT`	A key idea or theory you must know for the exam
`CONCLUSION`	What the practical proves — write this in your exam answer

All code uses Selenium with Python. ChromeDriver must be installed and added to system PATH before running any practical.

1. Selenium Environment Setup

COMMANDS — Run these in Terminal / Command Prompt

lightblue Command / Method	What it does
`pip install selenium`	Installs the Selenium library for Python
`pip install webdriver-manager`	Installs the WebDriver Manager (optional helper)
`python --version`	Checks that Python is installed correctly
`pip --version`	Checks that pip (Python package manager) is working

CODE — Write this in a file called `practical1.py`

```
from selenium import webdriver      # Import the Selenium
    WebDriver module

driver = webdriver.Chrome()         # Open Google Chrome browser
     automatically
driver.get('https://www.google.com') # Tell browser to go to
    Google
print(driver.title)                 # Print page title (should
    print: Google)
driver.quit()                       # Close the browser when
    done
```

Save the file as `practical1.py` and run it using: `python practical1.py`

WebDriver successfully opens Chrome browser, navigates to a URL, and fetches the page title — confirming the Selenium setup is correct.

2. Browser Navigation

COMMANDS — Selenium navigation methods used in code

lightblue Command / Method	What it does
`driver.get(url)`	Opens the given URL in the browser (like typing in address bar)
`driver.back()`	Goes to the previous page (like pressing the Back button)
`driver.forward()`	Goes to the next page (like pressing the Forward button)
`driver.refresh()`	Reloads the current page (like pressing F5)
`time.sleep(2)`	Pauses the script for 2 seconds so you can see what is happening

CODE — Write this in practical2.py

```
from selenium import webdriver     # Import Selenium
import time                        # Import time module (used
   for pause)

driver = webdriver.Chrome()        # Open Chrome browser

driver.get('https://google.com')   # Step 1: Open Google
time.sleep(2)                      # Wait 2 seconds

driver.get('https://wikipedia.org') # Step 2: Navigate to
   Wikipedia
time.sleep(2)                        # Wait 2 seconds

driver.back()      # Step 3: Go back to Google
driver.forward()   # Step 4: Go forward to Wikipedia
driver.refresh()   # Step 5: Refresh the current page

driver.quit()      # Close the browser
```

Navigation commands (get, back, forward, refresh) allow Selenium to control browser movement just like a real user clicking buttons.

3. Locators — ID, XPath, CSS Selector

Locators are used to FIND elements (buttons, text boxes, links) on a web page. Selenium supports multiple locator strategies.

COMMANDS — Locator types

lightblue Command / Method	What it does
`By.ID`	Find element by its unique id attribute — fastest and most reliable
`By.XPATH`	Find element using XPath expression — very flexible, works anywhere
`By.CSS_SELECTOR`	Find element using CSS selector syntax — fast and common
`By.NAME`	Find element by its name attribute
`By.CLASS_NAME`	Find element by its CSS class name
`.send_keys('abc')`	Type the text 'abc' into the found element
`.click()`	Click on the found element

CODE — Write this in `practical3.py`

```
from selenium import webdriver
from selenium.webdriver.common.by import By  # Import By class
    for locators

driver = webdriver.Chrome()
driver.get('file:///path/demo.html')  # Open a local HTML file

# Locator 1: Using ID
driver.find_element(By.ID, 'username').send_keys('admin')

# Locator 2: Using XPath
driver.find_element(By.XPATH, '//button').click()

# Locator 3: Using CSS Selector
driver.find_element(By.CSS_SELECTOR, '#username').send_keys('
    test')

driver.quit()
```

Locators (ID, XPath, CSS) are used to uniquely identify HTML elements. Each has its own use case. ID is fastest; XPath is most flexible.

4. Form Automation

Form Automation means writing a script that automatically fills in text fields, selects options, and submits a form — without human involvement.

COMMANDS — Methods used to automate a form

lightblue Command / Method	What it does
`.send_keys('value')`	Types the given value into a text field
`.click()`	Clicks a button, radio button, or checkbox
`By.ID`	Finds the element using its HTML id attribute

CODE — Write this in practical4.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get('file:///path/form.html')  # Open the local form
    HTML file

# Fill text fields
driver.find_element(By.ID, 'name').send_keys('Rahul')
driver.find_element(By.ID, 'email').send_keys('rahul@gmail.com'
    )
driver.find_element(By.ID, 'password').send_keys('1234')

# Select radio button
driver.find_element(By.ID, 'male').click()   # Click the Male
    radio button

# Submit the form
driver.find_element(By.ID, 'submitBtn').click()  # Click Submit
     button

driver.quit()
```

Forms can be filled and submitted automatically using send_keys() for text input and click() for buttons and radio buttons.

5. Dropdown and Checkbox Handling

Dropdown: Uses the `Select` class from Selenium. Checkbox: Simply use `.click()` to toggle it on or off.

COMMANDS — Key methods

lightblue Command / Method	What it does
`Select(element)`	Wraps an HTML <select> element to allow easy selection
`.select_by_visible_text('t')`	Selects dropdown option by the text visible to the user
`.select_by_value('val')`	Selects dropdown option by its HTML value attribute
`.select_by_index(n)`	Selects dropdown option by its position (0 = first item)
`.click()`	Checks or unchecks a checkbox

CODE — Write this in practical5.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select  # Import
   Select for dropdowns

driver = webdriver.Chrome()
driver.get('file:///path/dropdown.html')

# -- DROPDOWN --
dropdown = driver.find_element(By.ID, 'city')  # Find the
   dropdown by ID
select = Select(dropdown)                       # Wrap with
   Select class
select.select_by_visible_text('Pune')           # Choose 'Pune'
    from the list

# -- CHECKBOX --
driver.find_element(By.ID, 'reading').click()  # Click checkbox
    to check it

driver.quit()
```

Dropdowns use Selenium's Select class with select_by_visible_text(). Checkboxes are handled with a simple click() call.

6. Alerts Handling

An alert is a pop-up box that appears in the browser. Selenium cannot interact with it directly — you must switch focus to the alert first using `switch_to.alert`.

COMMANDS — Alert handling methods

lightblue Command / Method	What it does
`driver.switch_to.alert`	Switches focus from the webpage to the alert pop-up
`alert.text`	Reads and returns the message shown in the alert box
`alert.accept()`	Clicks the OK button on the alert (accepts it)
`alert.dismiss()`	Clicks the Cancel button on the alert (dismisses it)
`alert.send_keys('t')`	Types text into a prompt-type alert box

CODE — Write this in practical6.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get('file:///path/alert.html')  # Open page that
    triggers an alert

# Trigger the alert
driver.find_element(By.XPATH, '//button').click()  # Click
    button to show alert

# Switch focus to the alert
alert = driver.switch_to.alert   # Move control to the alert
    box

# Read and accept the alert
print(alert.text)   # Print the alert message
alert.accept()      # Click OK to close the alert

driver.quit()
```

JavaScript alerts are handled by switching focus using switch_to.alert. You can then read the alert message, accept, or dismiss it.

7. Screenshot Capturing

Screenshots capture what the browser displays at a specific moment. Used for bug reporting, test evidence, and debugging failures.

COMMANDS — Screenshot methods

lightblue Command / Method	What it does
`driver.save_screenshot('f.png')`	Saves a full-page screenshot of the browser window
`element.screenshot('el.png')`	Saves a screenshot of only a specific element

CODE — Write this in practical7.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get('https://google.com')  # Open Google

# Screenshot 1: Full page screenshot
driver.save_screenshot('full.png')   # Saves entire browser
   window as full.png

# Screenshot 2: Element-only screenshot
element = driver.find_element(By.NAME, 'q')  # Find Google
   search box
element.screenshot('element.png')            # Save only the
   search box

driver.quit()
```

Selenium can capture the full browser window or individual elements as PNG images, which are helpful for debugging and test reporting.

8. Actions Class — Mouse & Keyboard

The `ActionChains` class allows advanced interactions like hovering, right-clicking, drag-and-drop, and key combinations — actions that `.click()` alone cannot do.

COMMANDS — ActionChains methods

lightblue Command / Method	What it does
`ActionChains(driver)`	Creates an Actions object linked to the current browser
`.move_to_element(el)`	Moves the mouse cursor over / hovers on the element
`.click(el)`	Left-clicks the element
`.context_click(el)`	Right-clicks the element (opens context menu)
`.double_click(el)`	Double-clicks the element
`.drag_and_drop(s,d)`	Drags element from source to destination
`.send_keys(Keys.CTRL,'a')`	Simulates pressing Ctrl+A keyboard shortcut
`.perform()`	REQUIRED: Executes all the queued actions above

CODE — Write this in practical8.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import
   ActionChains

driver = webdriver.Chrome()
driver.get('file:///path/actions.html')

actions = ActionChains(driver)  # Create ActionChains object

element = driver.find_element(By.ID, 'box')  # Find element
   with id='box'

# Action 1: Hover over the element
actions.move_to_element(element).perform()   # Hover then
   execute

# Action 2: Right-click the element
actions.context_click(element).perform()     # Right-click then
```

```
    execute

driver.quit()
```

ActionChains enables advanced mouse and keyboard interactions like hovering, right-clicking, and drag-and-drop, which go beyond basic click().

9. Web Table Handling

Web tables are HTML <table> elements with rows (<tr>) and cells (<td>). Selenium reads them row-by-row and cell-by-cell.

COMMANDS — Methods used to read tables

lightblue Command / Method	What it does
`find_elements(By.XPATH,'//table/tr')`	Gets ALL rows in the table as a list
`find_elements(By.TAG_NAME,'td')`	Gets ALL cells in a single row as a list
`cols[0].text`	Reads the text content of the first cell (index 0)
`len(rows)`	Gets the total number of rows in the table
`range(1, len(rows))`	Loop from row 1 (skip header row 0) to the last row

CODE — Write this in practical9.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get('file:///path/table.html')  # Open HTML file with a
    table

# Get all rows from the table
rows = driver.find_elements(By.XPATH, '//table/tr')  # Find all
     <tr> elements

# Loop through rows - start from 1 to skip the header row
for i in range(1, len(rows)):
    cols = rows[i].find_elements(By.TAG_NAME, 'td')  # Get all
        <td> in this row
    print(cols[0].text, cols[1].text, cols[2].text)  # Print
        col 1, 2, 3

driver.quit()
```

Web tables can be read row by row and cell by cell using find_elements(). XPath `//table/tr` fetches all rows, and TAG_NAME `td` fetches all cells within a row.

10. Wait Mechanisms

Modern websites load elements dynamically. If Selenium tries to find an element before it appears, it throws an error. Waits solve this problem.

COMMANDS — Types of Waits

lightblue Command / Method	What it does
`driver.implicitly_wait(10)`	IMPLICIT: Waits up to 10s for ANY element to appear globally
`WebDriverWait(driver, 10)`	EXPLICIT: Creates a wait object with 10 second timeout
`wait.until(EC.condition)`	Waits until a specific condition becomes true
`EC.element_to_be_clickable`	Condition: element is visible AND clickable
`EC.presence_of_element_located`	Condition: element appears in the DOM
`EC.visibility_of_element_located`	Condition: element is visible on screen
`time.sleep(3)`	STATIC: Pauses exactly 3 seconds (not recommended)

CODE — Write this in practical10.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as
    EC

driver = webdriver.Chrome()

# -- IMPLICIT WAIT (apply once, affects all find_element calls)
    --
driver.implicitly_wait(10)  # Wait up to 10 sec for any element
    to load

driver.get('https://google.com')

# -- EXPLICIT WAIT (apply to a specific element) --
wait = WebDriverWait(driver, 10)   # Wait object with 10 sec
    timeout

# Wait until the search box is clickable, then type in it
```

```
element = wait.until(EC.element_to_be_clickable((By.NAME, 'q'))
    )
element.send_keys('Selenium')       # Once ready, type 'Selenium
    '

driver.quit()
```

Implicit Wait applies globally to all elements. Explicit Wait targets a specific element and waits for a custom condition. Explicit waits are more reliable and recommended over implicit waits.

Quick Exam Reference Sheet

navyblue			
lightgrey 1	Environment Setup	`webdriver.Chrome()` `driver.get()` `driver.quit()`	Always end with `driver.quit()`
2	Browser Navigation	`driver.back()` `driver.forward()` `driver.refresh()`	Use `time.sleep()` to pause
lightgrey 3	Locators	`By.ID` `By.XPATH` `By.CSS_SELECTOR`	ID is fastest, XPath is flexible
4	Form Automation	`.send_keys('val')` `.click()`	Use click() for buttons/radio
lightgrey 5	Dropdown & Checkbox	`Select(element)` `.select_by_visible_text()`	Import Select from support.ui
6	Alerts	`driver.switch_to.alert` `alert.accept()`	Must switch_to before accept
lightgrey 7	Screenshot	`driver.save_screenshot()` `element.screenshot()`	Two types: full page & element
8	Actions Class	`ActionChains(driver)` `.move_to_element().perform()`	Always call .perform() at end
lightgrey 9	Web Table	`find_elements(By.XPATH,` `'//table/tr')`	Skip header with range(1,n)
10	Wait Mechanisms	`implicitly_wait(10)` `WebDriverWait + EC`	Explicit > Implicit for reliability

All the best for your exam! — SVTT Quick Guide